

Maddie Masiello
William Taing

Lab Group A3
EE 329-03 S'25
2025-Apr-23

Introduction:

The code for the A3 works as intended without any errors or bugs in the code. The added extra credit features of inputting time right to left, led flash once per second during countdown, and led dance at the end of timer were also all successfully implemented. If the star or reset button pressed very quickly, the timer will not return to the time input state, but to the greeting screen. If the star or reset button is held for a short time, then the timer will return directly to the time input state, presumably skipping over the greeting screen. The exact functionality of the star was not explicitly stated in the instructions. The timing and delay functions throughout the project can be decreased to the minimum necessary time to increase the speed and response of the program. We learned that for any project, especially one including the LCD, timing is extremely important to ensure data being written and read correctly.

Link to YouTube Video Presentation: <https://youtu.be/qgfxiS40R3U>

(Date of posted video: 4/23/25)

(Length: 1:56)

Obtained Data:

Figure A3(a): MCU-keypad-LED wiring diagram

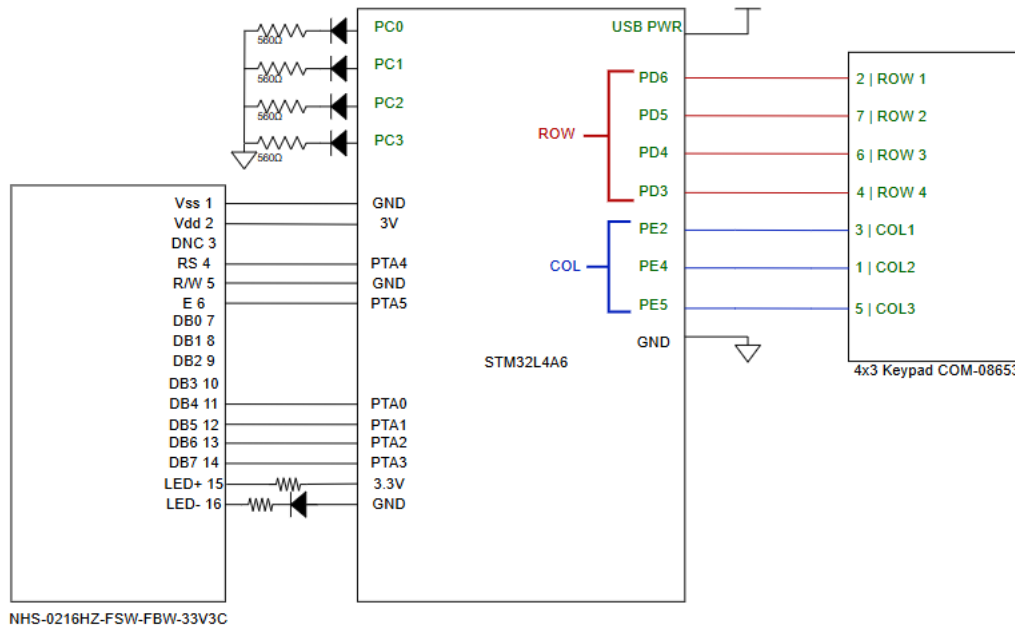


Figure A3(b) Pullup/pulldown values, with document and page number of the data Datasheet - STM32L4A6xG: Pg. 176 Table 70. I/O static characteristics

$R_{\text{Pull-up/Pull-down}} = 40\text{k}\Omega$ typical ($25\text{k}\Omega_{\text{min}} - 55\text{k}\Omega_{\text{max}}$)

Figure A3(c) Calibrated Delay Loop Timing

Entered Time	Actual Time	Loop Delay Length	Delay Loop Iterations
00:30	00:30.01	1000	1000
01:30	01:29:98	1000	1000
03:00	02:59:70	1000	1000

Pseudocode:

Main.c

- Initialize SysTick, keypad, lcd, and led pins
- Set led pins to 0
- Begin state
- STATE_IDLE_INIT
 - Delay for lcd startup
 - Clear lcd
 - Write welcome message and instructions
- STATE_IDLE
 - Wait for * set key press
- STATE_ENTRY_INIT

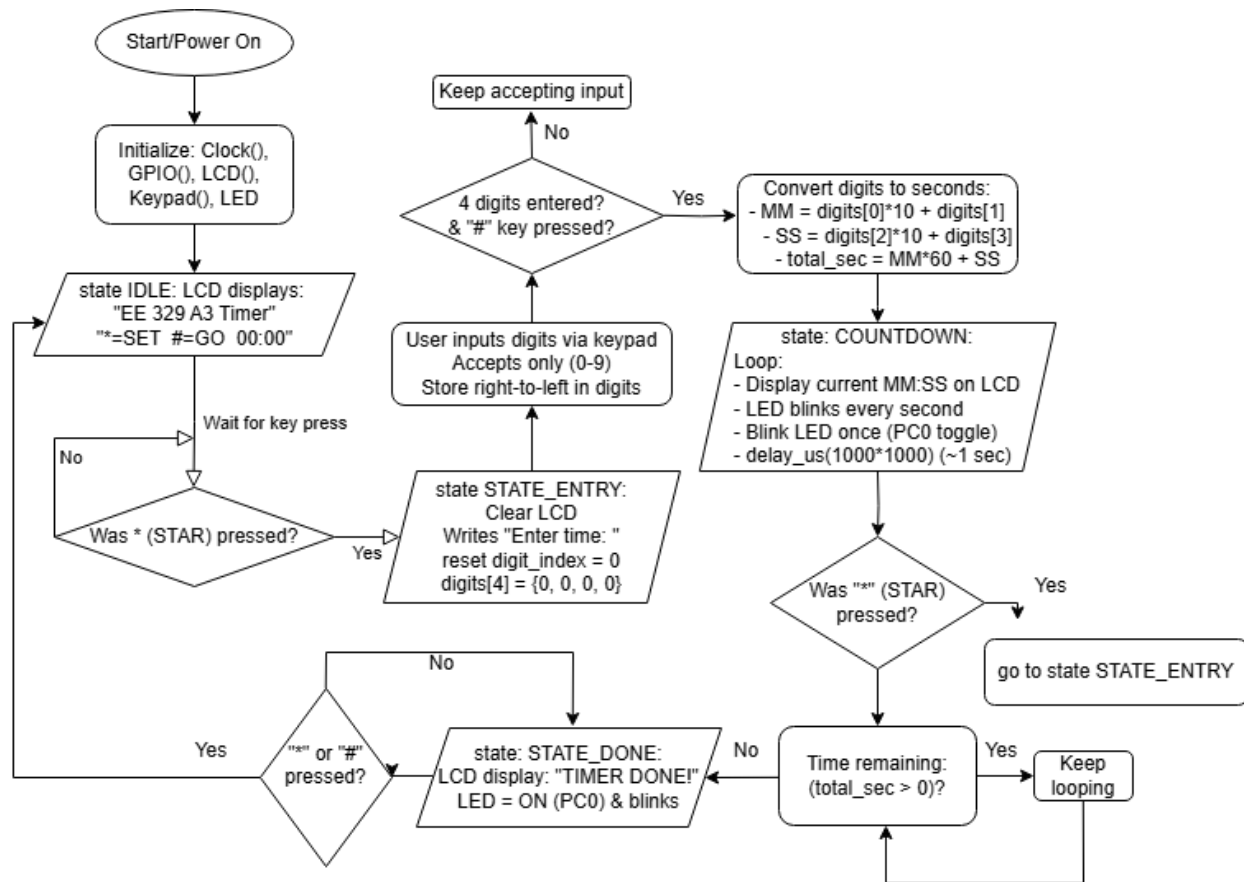
- If keypress is number
 - Write digits to lcd right to left
- If keypress is # go
 - Set state to STATE_COUNTDOWN
 - break
- If keypress is * set key press
 - Set state to STATE_ENTRY_INIT
 - break
- STATE_COUNTDOWN
 - While time>0
 - Convert invalid time to valid time
 - For(i=0; i<1000; i++)
 - Delay 1000us = 1ms
 - If keypress is * set key press
 - Set state to STATE_ENTRY_INIT
 - Break
 - Blink led
 - Time--
 - Set state to STATE_DONE
 - Break
- STATE_DONE
 - Write “Timer Done!”
 - Flash led
 - If keypress is * set or # go
 - Turn off led
 - Set state to STATE_IDLE_INIT
 - break
- DEFAULT
 - Set state to STATE_IDLE_INIT
 - break

LCD.c

- LCD_GPIO CONFIG
 - Setup lcd pins to output, push pull, very high speed, and no pullup/down
- LCD_init
 - Call LCD_GPIO CONFIG
 - Wait for lcd powerup
 - Wake up lcd then delay
 - Set to 4 bit mode then delay
 - Set functions 4 bit and 2 lines then delay

- Set cursor move direction then delay
- Set display, cursor, and blink on then delay
- Set entry mode increment cursor and no shift then delay
- Clear display then delay
- Restet cursor then delay
- LCD_pulse_ENA
- LCD_4b_command
- LCD_command
- LCD_write_char
- LCD_write_string
 - Call LCD_write_char for each letter in string
 - Delay
- LCD_clear
 - Clear display command
 - Delay
- LCD_set_cursor
 - Set cursor to given row and col
- DisplayTime
 - Set cursor then write digits
 - Repeat for each digit in time array
 - Write : between MM and SS

Pseudocode Flow Chart:



Appendix:

Formatted Source Code: main.h

```

/*****
 * EE 329 A2 KEYPAD INTERFACE
 *****/
 * @file      : main.h
 * @brief     : keypad configuration and debounced detection of keypresses
 * project    : EE 329 S'23 Assignment A2
 * authors    : Maddie Masiello
 * version    : 1
 * date       : 230413
 * compiler   : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target     : NUCLEO-L4A6ZG
 * clocks     : 4 MHz MSI to AHB2
 * @attention : (c) 2023 STMicroelectronics. All rights reserved.
 *****/

 * KEYPAD PLAN :
 * set columns as outputs, rows as inputs w pulldowns
 * loop:

```

```

* drive all columns HI read all rows
* if any row N is HI
*   set all columns LO
*   drive each column M HI alone
*   read row N until HI is pressed key loc'n = N, M
* key value = 3N+M+1 for 1..9, special case for *,0,#
*****
* KEYPAD WIRING 4 ROWS 3 COLS (pinout NUCLEO-L4A6ZG = L496ZG)
*   peripheral - Nucleo I/O
* keypad 1 COL 2 - PF13= CN10- 2 - OUT
* keypad 2 ROW 1 - PF0 = CN9 -21 - IN, PD ON
* keypad 3 COL 1 - PF12= CN7 -20 - OUT
* keypad 4 ROW 4 - PF3 = CN8 -14 - IN, PD ON
* keypad 5 COL 3 - PF14= CN10- 8 - OUT
* keypad 6 ROW 3 - PF2 = CN9 -17 - IN, PD ON
* keypad 7 ROW 2 - PF1 = CN9 -19 - IN, PD ON
*****
* REVISION HISTORY
* 230413 - Masiello - initial version
*****
* TODO
*
*****
*/

#ifndef __MAIN_H
#define __MAIN_H

#ifdef __cplusplus
extern "C"
{
#endif

/* Includes -----*/
#include "stm32l4xx_hal.h"

/* Exported functions prototypes -----*/
*/

void SystemClock_Config(void);
void Error_Handler(void);

#ifdef __cplusplus
}
#endif
#endif

```

Delay.h:

```
#ifndef INC_DELAY_H_
#define INC_DELAY_H_

#include "stm32l4xx.h"

void SysTick_Init(void);
void delay_us(uint32_t time_us);

#endif /* INC_DELAY_H_ */
```

Keypad.h

```
/* USER CODE BEGIN Header */
/*****
 * EE 329 A2 KEYPAD INTERFACE
 *****/
* @file      : keypad.h
* @brief     : Keypad GPIO pin mapping and constants for a 4x3 matrix keypad
* project    : EE 329 S'23 Assignment 2
* authors    : Maddie Masiello - mmasiell@calpoly.edu
* version    : 0.1
* date       :
* compiler   : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
* target     : NUCLEO-L4A6ZG
* clocks     : 4 MHz MSI to AHB2
* @attention : (c) 2023 STMicroelectronics. All rights reserved.
*****/
* KEY MAPPING:
* Column outputs: GPIOE pins PE2 (COL1), PE4 (COL2), PE5 (COL3)
* Row inputs:     GPIOD pins PD6 (ROW1), PD5 (ROW2), PD4 (ROW3), PD3 (ROW4)
*
* MATRIX LOGIC:
*      COL1  COL2  COL3
* R1   1      2      3
* R2   4      5      6
* R3   7      8      9
* R4   *      0      #
*
* Encoded key values: 1-9 as-is, '*'=10, '0'=11, '#'=12
* For LED display, key '0' is remapped to 0xF (CODE_ZERO)
*
* REVISION HISTORY
* 0.1
```

```

*****
*/

#ifndef INC_KEYPAD_H_
#define INC_KEYPAD_H_

#include "stm32l4xx_hal.h"

// ----- Boolean Constants -----
#define TRUE 1
#define FALSE 0

// ----- Keypad GPIO Mapping -----
// COL_PORT = GPIOE: PE2 (COL1), PE4 (COL2), PE5 (COL3)
// ROW_PORT = GPIOD: PD6 (ROW1), PD5 (ROW2), PD4 (ROW3), PD3 (ROW4)

#define COL_PORT GPIOE
#define COL_PORTEN RCC_AHB2ENR_GPIOEEN
#define ROW_PORT GPIOD
#define ROW_PORTEN RCC_AHB2ENR_GPIODEN

// Columns (in logical order: COL1, COL2, COL3)
#define COL_PINS (GPIO_PIN_2 | GPIO_PIN_4 | GPIO_PIN_5)
#define COL_PINSPOS1 2
#define COL_PINSPOS2 4
#define COL_PINSPOS3 5
#define BIT0_COL GPIO_PIN_2 // First bit for column scanning
// Rows (in logical order: ROW1, ROW2, ROW3, ROW4)
#define ROW_PINS (GPIO_PIN_6 | GPIO_PIN_5 | GPIO_PIN_4 | GPIO_PIN_3)
#define ROW_PINSPOS1 6
#define ROW_PINSPOS2 5
#define ROW_PINSPOS3 4
#define ROW_PINSPOS4 3
#define BIT0_ROW GPIO_PIN_6 // First bit for row scanning (ROW1 = PD6)
// Compatibility define for legacy use
#define BIT0 BIT0_ROW
// ----- Keypad Geometry -----
#define NUM_COLS 3
#define NUM_ROWS 4

// ----- Timing -----
#define SETTLE 1900 // ~20ms settle delay for signal to stabilize

// ----- Keypad Key Encoding -----
// Encoded values: 1-9 = keys 1-9
// '*' = 10, '#' = 12, '0' = 11 → remapped to 0xF for LED

```



```

#define NO_KEYPRESS -1
#define KEY_ZERO 11
#define CODE_ZERO 0xF // 0xF = 15 → 4-bit LED all ON for '0'

// ----- Function Prototypes -----
void Keypad_Config(void);
int Keypad_DebouncedKey(void);
void Keypad_UpdateKey(void);

#endif /* INC_KEYPAD_H_ */

```

Lcd.h:

```

#ifndef INC_LCD_H_
#define INC_LCD_H_

#include "stm32l4xx.h"
#include "delay.h"

#define LCD_PORT GPIOA
#define LCD_RS GPIO_PIN_4 // PA4
#define LCD_EN GPIO_PIN_5 // PA5
#define LCD_DB4 GPIO_PIN_0 // PA0
#define LCD_DB5 GPIO_PIN_1 // PA1
#define LCD_DB6 GPIO_PIN_2 // PA2
#define LCD_DB7 GPIO_PIN_3 // PA3
#define LCD_DATA_BITS (LCD_DB4 | LCD_DB5 | LCD_DB6 | LCD_DB7)
#define LCD_PORTEN RCC_AHB2ENR_GPIOAEN // port enable mask
#define RSp0s 4
#define ENpos 5
#define DB4pos 0
#define DB5pos 1
#define DB6pos 2
#define DB7pos 3

void LCD_init(void);
void LCD_command(uint8_t cmd);
void LCD_write_char(uint8_t letter);
void LCD_write_string(char *str);
void LCD_4b_command(uint8_t command);
void LCD_set_cursor(uint8_t pos[2]);
void LCD_clear(void);
void DisplayTime(uint8_t *digits);

```

```
#endif /* INC_LCD_H_ */
```

Main.c

```
/* USER CODE BEGIN Header */
/*****
 * EE 329 A3 TIMER
 *****/
* @file      : main.c
* @brief     : Implements MM:SS countdown timer using keypad and LCD
* project    : EE 329 S'25 Assignment A3
* authors    : Maddie Masiello - mmasiell@calpoly.edu
* version    : 1.0
* date       : 2025-04-19
* compiler   : STM32CubeIDE v1.12.0
* target     : NUCLEO-L4A6ZG
* clocks     : 4 MHz MSI to AHB2
* @attention : (c) 2025 STMicroelectronics. All rights reserved.
*****/
* OVERVIEW:
* This project implements a countdown timer where time is entered via a 4x3 keypad
* and displayed on a 16x2 LCD. The timer counts down in MM:SS format and flashes
* an LED on PC0 when the time reaches zero. Pressing * restarts the time entry
* process, and # begins or ends the countdown.
*****/
* PINOUT:
* Keypad COLs: GPIOD 0-2 (outputs)
* Keypad ROWs: GPIOD 3-6 (inputs, pulldown)
* LCD Data: GPIOA 0-3 (D4-D7), GPIOA 4 (RS), GPIOA 5 (EN)
* LED: PC0 (output)
*****/
/* USER CODE END Header */

#include "main.h"
#include "keypad.h"
#include "lcd.h"
#include "delay.h"

// Define timer states
typedef enum
{
    STATE_IDLE_INIT,
    STATE_IDLE,
```

```

    STATE_ENTRY_INIT,
    STATE_ENTRY,
    STATE_READY,
    STATE_COUNTDOWN,
    STATE_DONE
} TimerState;

/*****
 * Timer digit storage: MM:SS format [min10, min1, sec10, sec1]
 * - MM:SS is stored as 4 separate digits in an array of 4 uint8_t values.
 * - The digits are updated in the countdown loop and displayed on the LCD.
 * - The timer starts from the entered time and counts down to zero.
 *****/
uint8_t digits[4] = {0, 0, 0, 0}; // MM:SS format
uint8_t digit_index = 0;
TimerState state = STATE_IDLE_INIT;

void SystemClock_Config(void); // Configure system clock

/*****
 * main(): Main function for the countdown timer program
 * - Initializes, configures the system clock, and enters the main loop.
 * - Does state transitions and user input from the keypad.
 *****/
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    SysTick_Init();
    Keypad_Config();
    LCD_init();

    RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOCEN);
    GPIOC->MODER &= ~(GPIO_MODER_MODE0 | GPIO_MODER_MODE1 |
                      GPIO_MODER_MODE2 | GPIO_MODER_MODE3);
    GPIOC->MODER |= (GPIO_MODER_MODE0_0 | GPIO_MODER_MODE1_0 |
                    GPIO_MODER_MODE2_0 | GPIO_MODER_MODE3_0);
    GPIOC->OTYPER &= ~(GPIO_OTYPER_OT0 | GPIO_OTYPER_OT1 |
                      GPIO_OTYPER_OT2 | GPIO_OTYPER_OT3);
    GPIOC->PUPDR &= ~(GPIO_PUPDR_PUPD0 | GPIO_PUPDR_PUPD1 |
                      GPIO_PUPDR_PUPD2 | GPIO_PUPDR_PUPD3);
    GPIOC->OSPEEDR |= ((3 << GPIO_OSPEEDR_OSPEED0_Pos) |
                       (3 << GPIO_OSPEEDR_OSPEED1_Pos) |
                       (3 << GPIO_OSPEEDR_OSPEED2_Pos) |

```

```

        (3 << GPIO_OSPEEDR_OSPEED3_Pos));
GPIOC->BRR = (GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3);

delay_us(2000);
uint8_t entry_lock = 0;

// LED = PC0

// infinite loop to run the timer program
while (1)
{
    delay_us(10000);
    Keypad_UpdateKey();

    int key = Keypad_DebouncedKey(); // Check for keypresses

    // switch statement to handle different states of the timer
    switch (state)
    {

        // STATE_IDLE_INIT: initializes the timer and displays welcome message
        case STATE_IDLE_INIT:
            delay_us(10000);
            LCD_clear(); // Clear LCD screen

            LCD_write_string("EE 329 A3 TIMER");

            LCD_set_cursor((uint8_t[]){0, 1}); // Set cursor to second line
            LCD_write_string("*=SET #=GO 00:00");

            DisplayTime((uint8_t[]){0, 0, 0, 0}); // Display initial time
            state = STATE_IDLE;                    // Set state to idle
            break;

        // STATE_IDLE: waiting for user input to set time
        case STATE_IDLE:
            // if key is pressed, check if it's a valid key
            if (key == 10)
            {
                state = STATE_ENTRY_INIT;
            }
            break;

        // STATE_ENTRY_INIT: initializes the digits array to 0 and
        // displays "Enter Time:" on the lcd
    }
}

```

```

case STATE_ENTRY_INIT:

    // for loop that initializes the digits array to 0
    for (int i = 0; i < 4; i++)
        digits[i] = 0;
    LCD_clear(); // clear the lcd
    LCD_write_string("Enter Time:");
    delay_us(400);
    DisplayTime(digits); // default display 00:00
    state = STATE_ENTRY;
    break;

// STATE_ENTRY: waiting for user to enter time
// uses an entry lock to prevent key repeats
// entry lock is set to 1 when a key is pressed and set to 0
//when no key is pressed
case STATE_ENTRY:

    // key unpressed, enable input
    if (key == NO_KEYPRESS)
    {
        entry_lock = 0;
    }

    // Disable key repeats
    if (entry_lock == 0)
    {
        // if the key is a digit (0-9) and the digit index
        // is not 0, store the digit
        if (key >= 0 && key <= 9)
        {
            // Shift digits to the left
            for (int i = 0; i < 3; i++)
                digits[i] = digits[i + 1];
            digits[3] = key; // Store the new digit in last pos
            DisplayTime(digits); // display time on the lcd
            entry_lock = 1; // set entry lock to 1
        }

        // if the key is 12 and the digit index is 0,
        //the state is set to countdown

        if (Keypad_DebouncedKey() == 12)
        {
            delay_us(400);

```

```

        state = STATE_COUNTDOWN; // POUND_KEY
        delay_us(400);
        break;
    }
    // if key is 10 (STAR_KEY), state = state_entry and reset digits
    // to 0, and display time on the lcd
    if (Keypad_DebouncedKey() == 10)
    {
        delay_us(400);
        state = STATE_ENTRY_INIT; // STAR_KEY
        delay_us(400);
        break;
    }
}
break;

// STATE_COUNTDOWN: countdown timer state that counts down from
//the entered time
case STATE_COUNTDOWN:
{
    // total_sec is calculated like this:
    // total_sec = (MM * 60) + SS =
    // ==((digits[0]*10 + digits[1]) * 60) + (digits[2]*10 + digits[3])
    // where digits[0] = tens of minutes, digits[1] = ones of minutes,
    // digits[2] = tens of seconds, digits[3] = ones of seconds
    // therefore, total_sec is the total number of seconds in the
    // entered time
    // for example, if the user enters 12:34,
    // total_sec = (1*10 + 2) * 60 + (3*10 + 4) = 1234 seconds
    int total_sec=((digits[0]*10+digits[1])*60)+(digits[2]*10+digits[3]);

    // while our total_sec is greater than 0, we will display
    // the time on the lcd
    while (total_sec > 0)
    {
        int m = total_sec / 60; // minutes
        int s = total_sec % 60; // seconds
        digits[0] = m / 10;      // tens of minutes
        digits[1] = m % 10;      // ones of minutes
        digits[2] = s / 10;      // tens of seconds
        digits[3] = s % 10;      // ones of seconds
        DisplayTime(digits);     // then display the time on the lcd

        key=Keypad_DebouncedKey(); // Check for keypresses
    }
}

```

```

        // for loop that counts down the time
        for (int i = 0; i < 1000; i++)
        {

            // if key is pressed, check if it's a valid key
            // NOTE: adjust delay value to change countdown speed
            delay_us(550);
            if (Keypad_DebouncedKey() == 10)
            {
                // STAR
                state = STATE_ENTRY_INIT; // enter entry state
                total_sec = 0;           // reset total_sec to 0
                break;
            }
            if(i>500){ // first half of second
                GPIOC->BRR = GPIO_PIN_0; // turn on LED
            }
            else{ // second half of second
                GPIOC->BSRR = GPIO_PIN_0; // turn off LED
            }
        }
        total_sec--; // decrement total_sec by 1

    }
    state = STATE_DONE; // set state to done
    break;
}

// STATE_DONE: this state is entered when the timer is done counting
// down and the LED is turned on, and a message is displayed on the lcd
case STATE_DONE:
    LCD_write_string("  TIMER DONE! "); // display done message on lcd
    LCD_set_cursor((uint8_t[]){0, 1}); // Set cursor to second line
    delay_us(100000);
    GPIOC->BSRR = GPIO_PIN_0;           // turn on LED
    delay_us(100000);
    GPIOC->BRR = GPIO_PIN_0;           // turn off LED
    Keypad_DebouncedKey(); // Check for keypresses
    if (key == 10 || key == 12)
    {
        // STAR [10] or POUND [12]
        GPIOC->BRR = GPIO_PIN_0; // turn off LED
        state = STATE_IDLE_INIT; // set state to idle
    }
    break;

// end of case statement, default case to set state to idle
default:

```

```

        state = STATE_IDLE_INIT;
        break;
    }
}

void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage */
    if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure. */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_MSI;
    RCC_OscInitStruct.MSIState = RCC_MSI_ON;
    RCC_OscInitStruct.MSICalibrationValue = 0;
    RCC_OscInitStruct.MSIClockRange = RCC_MSIRANGE_6;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK |
RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_MSI;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
    {
        Error_Handler();
    }
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

```



```

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /*User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
}

#ifdef USE_FULL_ASSERT
void assert_failed(uint8_t *file, uint32_t line)
{
}
#endif

```

Delay.c:

```

#include "delay.h"

/* ----- SysTick_Init(void)-----
 * SysTick_Init() initializes the SysTick timer for 1 microsecond delay.
 * ----- */
void SysTick_Init(void)
{
    SysTick->CTRL |= (SysTick_CTRL_ENABLE_Msk | SysTick_CTRL_CLKSOURCE_Msk);
    SysTick->CTRL |= SysTick_CTRL_TICKINT_Msk;
}

/* ----- delay_us(uint32_t us)-----
 * delay_us() generates a delay in microseconds using the DWT cycle counter.
 * It uses the SystemCoreClock to calculate the number of ticks needed for the
 * specified delay.
 * ----- */
void delay_us(uint32_t us)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
    uint32_t start = DWT->CYCCNT;

```

```

uint32_t ticks = us * (SystemCoreClock / 1000000);
while ((DWT->CYCCNT - start) < ticks)
    ;
}

```

Keypad.c:

```

/* USER CODE BEGIN Header */
/*****
 * EE 329 A2 KEYPAD INTERFACE
 *****/
* @file      : keypad.c
* @brief     : keypad configuration and debounced detection of keypresses
* project    : EE 329 S'23 Assignment A2
* authors    : Maddie Masiello, Tatiana Ribay
* version    : 0.1
* date       :
* compiler   : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
* target     : NUCLEO-L4A6ZG
* clocks     : 4 MHz MSI to AHB2
* @attention : (c) 2023 STMicroelectronics. All rights reserved.
*****/
* Keypad Wiring 4 ROWS 3 COLS (pinout NUCLEO-L4A6ZG = L496ZG)
*****/
* REVISION HISTORY
* 0.1
*****/
*/

#include "keypad.h"
#include "delay.h"

int current_key = NO_KEYPRESS;
int last_key = NO_KEYPRESS;
int last_key_pressed_time = -1;

/* ----- Keypad_Config()-----
 * Keypad_Config() initializes the GPIO pins for the keypad interface.
 * The keypad is a 4x3 matrix with 4 rows and 3 columns.
 * The rows are configured as inputs with pull-down resistors,
 * and the columns are configured as outputs.
 * The function enables the GPIOF clock and sets the appropriate mode, type,
 * speed, and pull-up/pull-down settings for the pins.
 * ----- */
void Keypad_Config(void){

```

```

RCC->AHB2ENR |= (COL_PORTEN | ROW_PORTEN); // Enable rol and col registers
COL_PORTEN->MODER &= ~(0b11<<(2*COL_PINSPOS1) |
0b11<<(2*COL_PINSPOS2) |
0b11<<(2*COL_PINSPOS3));
ROW_PORTEN->MODER &= ~(0b11<<(2*ROW_PINSPOS1) |
0b11<<(2*ROW_PINSPOS2) |
0b11<<(2*ROW_PINSPOS3) |
0b11<<(2*ROW_PINSPOS4)); // reset

COL_PORTEN->MODER |= (0b01<<(2*COL_PINSPOS1) |
0b01<<(2*COL_PINSPOS2) |
0b01<<(2*COL_PINSPOS3));
ROW_PORTEN->MODER |= (0b01<<(2*ROW_PINSPOS1) |
0b01<<(2*ROW_PINSPOS2) |
0b01<<(2*ROW_PINSPOS3) |
0b01<<(2*ROW_PINSPOS4)); // output

COL_PORTEN->OTYPER &= ~(0b1<<(COL_PINSPOS1) |
0b1<<(COL_PINSPOS2) |
0b1<<(COL_PINSPOS3));
ROW_PORTEN->OTYPER &= ~(0b1<<(ROW_PINSPOS1) |
0b1<<(ROW_PINSPOS2) |
0b1<<(ROW_PINSPOS3) |
0b1<<(ROW_PINSPOS4)); // push/pull

COL_PORTEN->OSPEEDR |= (0b11<<(2*COL_PINSPOS1) |
0b11<<(2*COL_PINSPOS2) |
0b11<<(2*COL_PINSPOS3));
ROW_PORTEN->OSPEEDR |= (0b11<<(2*ROW_PINSPOS1) |
0b11<<(2*ROW_PINSPOS2) |
0b11<<(2*ROW_PINSPOS3) |
0b11<<(2*ROW_PINSPOS4)); // vhi spd

COL_PORTEN->PUPDR &= ~(0b11<<(2*COL_PINSPOS1) |
0b11<<(2*COL_PINSPOS2) |
0b11<<(2*COL_PINSPOS3));
ROW_PORTEN->PUPDR &= ~(0b11<<(2*ROW_PINSPOS1) |
0b11<<(2*ROW_PINSPOS2) |
0b11<<(2*ROW_PINSPOS3) |
0b11<<(2*ROW_PINSPOS4)); // reset

COL_PORTEN->PUPDR |= (0b10<<(2*COL_PINSPOS1) |
0b10<<(2*COL_PINSPOS2) |
0b10<<(2*COL_PINSPOS3));
ROW_PORTEN->PUPDR |= (0b10<<(2*ROW_PINSPOS1) |
0b10<<(2*ROW_PINSPOS2) |

```

```

0b10<<(2*ROW_PINSP0S3) |
0b10<<(2*ROW_PINSP0S4)); //no pupl
}

/* ----- Keypad_IsAnyKeyPressed()-----
 * Keypad_IsAnyKeyPressed() checks if any key on the keypad is pressed.
 * It drives all columns high and checks if any row is high.
 * If a key is pressed, it returns TRUE; otherwise, it returns FALSE.
 * No debounce, just looking for a key twitch.
 * ----- */
int Keypad_IsAnyKeyPressed(void)
{
    // drive all COLUMNS HI; see if any ROWS are HI
    // return true if a key is pressed, false if not
    // currently no debounce here - just looking for a key twitch
    COL_PORT->BSRR = COL_PINS;          // set all columns HI
    if ((ROW_PORT->IDR & ROW_PINS) != 0) // got a keypress!
        return TRUE;
    else
        return FALSE;
}

/* ----- Keypad_WhichKeyPressed()-----
 * Keypad_WhichKeyPressed() detects and encodes a pressed key at {row,col}
 * It assumes a previous call to Keypad_IsAnyKeyPressed() returned TRUE.
 * It verifies the Keypad_IsAnyKeyPressed() result (no debounce here),
 * determines which key is pressed, and returns the encoded key ID.
 * ----- */
int Keypad_WhichKeyPressed(void)
{
    int8_t iRow = 0, iCol = 0;
    int8_t bGotKey = 0;
    int iKey = NO_KEYPRESS;

    const uint16_t colMap[NUM_COLS] = {GPIO_PIN_2, GPIO_PIN_4, GPIO_PIN_5};
    const uint16_t rowMap[NUM_ROWS] = {GPIO_PIN_6, GPIO_PIN_5, GPIO_PIN_4,
GPIO_PIN_3};

    // Step 1: Set all columns high
    COL_PORT->BSRR = COL_PINS;

    for (iRow = 0; iRow < NUM_ROWS; iRow++)
    {
        if (ROW_PORT->IDR & rowMap[iRow])
        {
            // Step 2: Pull all columns low

```

```

    COL_PORT->BRR = COL_PINS;

    // Step 3: Raise one column at a time
    for (iCol = 0; iCol < NUM_COLS; iCol++)
    {
        COL_PORT->BSRR = colMap[iCol]; // Raise one column
        delay_us(50);                  // Let signals settle

        if (ROW_PORT->IDR & rowMap[iRow])
        {
            bGotKey = 1;
            break;
        }

        COL_PORT->BRR = colMap[iCol]; // Reset column
    }
    if (bGotKey)
        break;
}

if (bGotKey)
{
    iKey = (iRow * NUM_COLS) + iCol + 1;
    if (iKey == KEY_ZERO)
        iKey = CODE_ZERO;
}

return iKey;
}

/*****
 * Keypad_DebouncedKey(): Detects a keypress with debounce
 * - Reads the keypad to check which key is pressed.
 *****/
void Keypad_UpdateKey(void)
{
    if (Keypad_IsAnyKeyPressed())
    {
        int key = Keypad_WhichKeyIsPressed(); // Get the pressed key
        if (key != last_key)
        {
            // If the key is different from the
last key
            last_key = key; // Update last key
            last_key_pressed_time = HAL_GetTick(); // Store the time of the last key
press

```

```

    }
    else if ((HAL_GetTick() - last_key_pressed_time) > 50)
    {
        current_key = (last_key == 15) ? 0 : last_key;
    }
}
else
{
    last_key = NO_KEYPRESS;
    current_key = NO_KEYPRESS;
}
}

/* ----- Keypad_DebouncedKey() ----- */
int Keypad_DebouncedKey(void)
{
    // Call Keypad_UpdateKey() to check for keypresses
    Keypad_UpdateKey();
    delay_us(400);
    return current_key;
}

```

lcd.c:

```

#include "lcd.h"
/* USER CODE BEGIN Header */
/*****
 * EE 329 A3 TIMER
 *****/
* @file      : lcd.c
* @brief     : Implements LCD control functions for EE 329 Assignment A3
* project    : EE 329 S'25 Assignment A3
* authors    : Maddie Masiello - mmasiell@calpoly.edu, Will Taing
* version    : 1.0
* date       : 2025-04-19
* compiler   : STM32CubeIDE v1.12.0
* target     : NUCLEO-L4A6ZG
* clocks     : 4 MHz MSI to AHB2
* @attention : (c) 2025 STMicroelectronics. All rights reserved.
*****/
* OVERVIEW:
* This file contains the implementation of functions to control an LCD display
* in 4-bit mode. It includes GPIO configuration, initialization, and utility

```

```

* functions for writing characters, strings, and setting the cursor position.
*****
*****/
/* USER CODE END Header */

/*****
* LCD_GPIO_Config(): Configures the GPIO pins for the LCD display.
* - Sets the mode to output for the data and control pins.
* - Sets the output type to push-pull.
* - Sets the speed to high for the data and control pins.
*****/
void LCD_GPIO_Config(void)
{
    RCC->AHB2ENR |= (LCD_PORTEN); //Enable LCD GPIO Register
    LCD_PORT->MODER &= ~(0b11<<(2*DB4pos) |
    0b11<<(2*DB5pos) |
    0b11<<(2*DB6pos) |
    0b11<<(2*DB7pos) |
    0b11<<(2*RSpos) |
    0b11<<(2*ENpos)); //reset
    LCD_PORT->MODER |= (0b01<<(2*DB4pos) |
    0b01<<(2*DB5pos) |
    0b01<<(2*DB6pos) |
    0b01<<(2*DB7pos) |
    0b01<<(2*RSpos) |
    0b01<<(2*ENpos)); //output
    LCD_PORT->OTYPER &= ~(0b1<<(DB4pos) |
    0b1<<(DB5pos) |
    0b1<<(DB6pos) |
    0b1<<(DB7pos) |
    0b1<<(RSpos) |
    0b1<<(ENpos)); //psh/pull
    LCD_PORT->OSPEEDR |= (0b11<<(2*DB4pos) |
    0b11<<(2*DB5pos) |
    0b11<<(2*DB6pos) |
    0b11<<(2*DB7pos) |
    0b11<<(2*RSpos) |
    0b11<<(2*ENpos)); //vhi spd
    LCD_PORT->PUPDR &= ~(0b11<<(2*DB4pos) |
    0b11<<(2*DB5pos) |
    0b11<<(2*DB6pos) |
    0b11<<(2*DB7pos) |
    0b11<<(2*RSpos) |
    0b11<<(2*ENpos)); //reset
    LCD_PORT->PUPDR |= (0b00<<(2*DB4pos) |

```

```

    0b00<<(2*DB5pos) |
    0b00<<(2*DB6pos) |
    0b00<<(2*DB7pos) |
    0b00<<(2*RSpos) |
    0b00<<(2*ENpos)); //nopupl
}

/*****
 * LCD_init(): Initializes the LCD display,
 * - Configures the GPIO pins for the LCD
 * - Sends the necessary commands to set up the LCD in 4-bit mode.
 * - Sets the display to 2 lines, 5x7 matrix, and clears the display.
 * - Sets the cursor to the home position.
 *****/
void LCD_init(void)
{
    LCD_GPIO_Config();
    delay_us(40000);
    //for loop that sends the wake-up command 3 times
    //for each iteration, the LCD is set to 4-bit mode
    //and the delay is set to 200us
    for (int idx = 0; idx < 3; idx++)
    {
        LCD_4b_command(0x30); //0x30 = 0011 0000
        delay_us(200);
    }
    //uses LCD_4b_command to set the LCD to 4-bit mode
    //and then sends the function set command to configure the LCD
    LCD_4b_command(0x20); //0x20 = 0010 0000
    delay_us(40);

    //uses LCD_command to set the display to 2 lines, 5x7 matrix
    LCD_command(0x28); // 0x28 = 0010 1000
    delay_us(300);

    LCD_command(0x10); // 0x10 = 0001 0000
    delay_us(300);

    LCD_command(0x0F); // 0x0F = 0000 1111
    delay_us(300);

    LCD_command(0x06); // 0x06 = 0000 0110
    delay_us(300);

    LCD_command(0x01); // 0x01 = 0000 0001
    delay_us(2000);

```



```

    LCD_command(0x80);          // 0x80 = 1000 0000
    delay_us(300);
}

/*****
 * LCD_pulse_ENA(): Pulses the ENABLE pin to latch the command/data.
 * - Sets the ENABLE pin high, waits for a short time, and then sets it low.
 *****/
void LCD_pulse_ENA(void)
{
    // Enable line sends command on falling edge
    // set to restore default then clear to trigger
    LCD_PORT->ODR |= (LCD_EN); // ENABLE = HI
    delay_us(5);              // TDDR > 320 ns
    LCD_PORT->ODR &= ~(LCD_EN); // ENABLE = LOW
    delay_us(5);              // low values flakey, see A3:p.1
}

/*****
 * LCD_4b_command(): Sends a command to the LCD in 4-bit mode.
 *****/
void LCD_4b_command(uint8_t command)
{
    // LCD command using high nibble only - used for 'wake-up' 0x30 commands
    LCD_PORT->ODR &= ~(LCD_DATA_BITS); // clear DATA bits
    LCD_PORT->ODR |= (command & 0x0F); // place nibble directly on PA0-PA3
    delay_us(5);
    LCD_pulse_ENA();
}

/*****
 * LCD_command(): Sends a command to the LCD in 4-bit mode.
 *****/
void LCD_command(uint8_t command)
{
    // send command to LCD in 4-bit instruction mode
    // HIGH nibble then LOW nibble, timing sensitive
    LCD_PORT->ODR &= ~(LCD_DATA_BITS); //isolate cmd bits
    LCD_PORT->ODR |= ((command >> 4) & LCD_DATA_BITS); //HIGH shifted low
    delay_us(5);
    LCD_pulse_ENA(); // latch HIGH NIBBLE

    LCD_PORT->ODR &= ~(LCD_DATA_BITS); // isolate cmd bits
    LCD_PORT->ODR |= (command & LCD_DATA_BITS); // LOW nibble
    delay_us(5);
}

```

```

    LCD_pulse_ENA(); // latch LOW NIBBLE
}

/*****
 * LCD_write_char(): Writes a character to the LCD display.
 *****/
void LCD_write_char(uint8_t letter)
{
    // calls LCD_command() w/char data; assumes all ctrl
    // bits set L0 in LCD_init()
    LCD_PORT->ODR |= (LCD_RS); // RS = HI for data to address
    delay_us(400);
    LCD_command(letter);      // character to print
    LCD_PORT->ODR &= ~(LCD_RS); // RS = L0
}

/*****
 * LCD_write_string(): Writes a string to the LCD display.
 * - Takes a null-terminated string and writes it to the LCD.
 *****/
void LCD_write_string(char *str)
{
    // writes each character in a null-terminated string to LCD
    while (*str)
    {
        LCD_write_char(*str++);
    }
    delay_us(400);
}

/*****
 * LCD_clear(): Clears the LCD display and sets the cursor to home position.
 *****/
void LCD_clear(void)
{
    // clears LCD display and sets cursor to home position
    LCD_command(0x01); // clear display command
    delay_us(10000);   // wait for clear command to complete
}

/*****
 * LCD_set_cursor(): Sets the cursor position on the LCD display.
 * - Takes a 2D array with the column and row position.
 *****/
void LCD_set_cursor(uint8_t pos[2])
{

```

```

    // sets LCD cursor to given (col, row)
    uint8_t row_offsets[] = {0x00, 0x40};
    uint8_t addr = 0x80 | (pos[0] + row_offsets[pos[1]]);
    LCD_command(addr);
}

/*****
 * DisplayTime(): Displays the current time on the LCD in MM:SS format
 * - Takes an array of 4 digits and displays them on the LCD.
 *****/
void DisplayTime(uint8_t *digits)
{ // points to the digits array

    // sets cursor to the appropriate position on the lcd and display the digits
    LCD_set_cursor((uint8_t[]){11, 1});
    LCD_write_char(digits[0] + '0');
    LCD_set_cursor((uint8_t[]){12, 1});
    LCD_write_char(digits[1] + '0');
    LCD_set_cursor((uint8_t[]){13, 1});
    LCD_write_char(':');
    LCD_set_cursor((uint8_t[]){14, 1});
    LCD_write_char(digits[2] + '0');
    LCD_set_cursor((uint8_t[]){15, 1});
    LCD_write_char(digits[3] + '0');
}

```